#4

CS 0447
Introduction to
Computer Programming

Memory and Addresses

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

Class announcements

- *Add/Drop* period *ends* ***??****!*
- Everyone joined slack? Links will/have? Expired
- Repeat after me:
  - Store copies from the CPU to memory
  - Load copies from memory to CPU

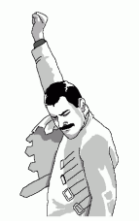
Variables, Loads, Stores

Memory addresses

- *Everything* in memory has an **address**
  - the position in memory **where it begins**
    - where its **first byte** is
  - this applies to variables, functions, objects, arrays etc.
- A super important concept:

every piece of data really has **two parts:**
an **address** and a **value**

- If you want to put a variable in memory…
  - first you need to figure out **what address to put it in**
  - this *extremely tedious* task is **handled by assemblers**
    - whew



Putting a variable in memory

- we can declare a global variable like this:

```
.data
    x: .word 4
```

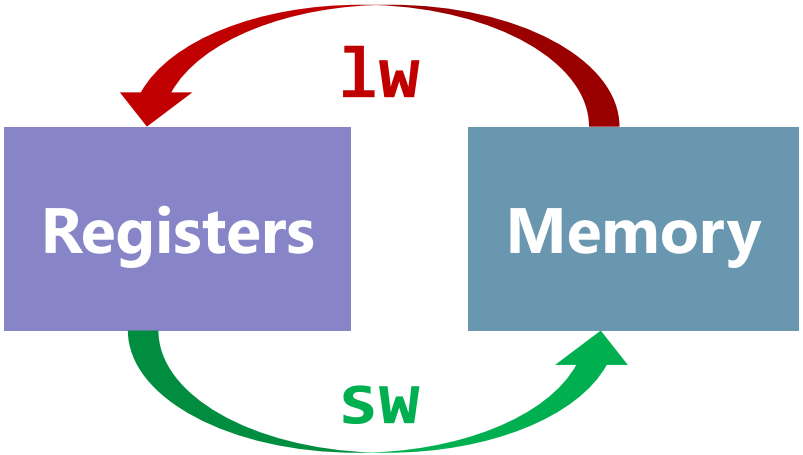
name — type — initial value

- the Java/C equivalent would be `static int x = 4;`
- `.data` says "I'm gonna declare variables"
  - you can declare as many as you want!
  - to go back to writing code, use `.text`
- if we assemble this little program and make sure **Tools > Show Labels Window** is checked, what do you see?
  - the assembler *gave the variable* that address
  - it'll do that for every variable

Load-store architectures

- In some architectures, *many* instructions can access memory
  - x86-64: `add [rsp-8], rcx`
    - adds the contents of **`rcx`** to the value at address **`rsp-8`**

- In a **load-store** architecture, **all** memory accesses are done with two kinds of instructions: loads and stores (like in MIPS)

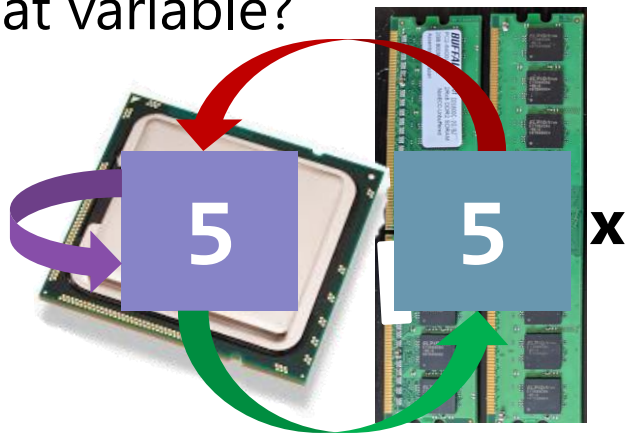


loads copy data **from** memory **into** CPU registers

stores copy data **from** CPU registers **into** memory

Operating on variables in memory (animated)

- we want to increment a variable that is **in memory**
  - where do values **have to be for the CPU to operate on them?**
  - what do we want the **overall outcome to be?**
- so, what **three steps** are needed to increment that variable?
  1. **load** the value from memory into a register
  2. **add 1** to the value in the register
  3. **store** the value back into memory
- **every variable access** works like this!!!
  - HLLs just hide this from you



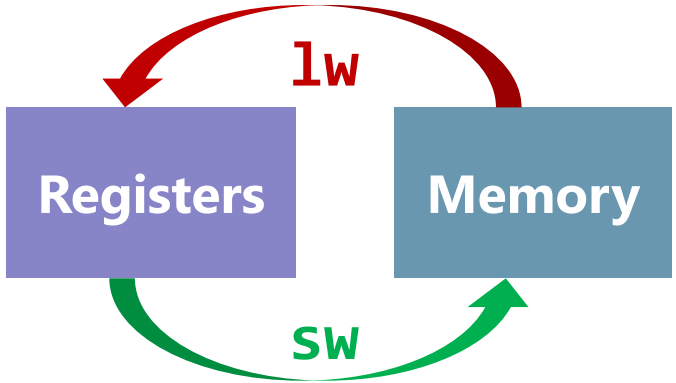
Accessing memory in MIPS

MIPS ISA: load and store instructions for words

- you can load and store entire 32-bit words with **lw** and **sw**
- the instructions look like this (variable names not important):

```
lw t1, x # loads from variable x into t1
sw t1, x # stores from t1 into variable x
```

- Ermm… In MIPS, **stores are written with the destination on the *right*. *!?***
  - well, you can remember it with this diagram…
  - the memory is "on the right" for both loads and stores



MIPS ISA: load and store instructions for words

- You can also load the 32-bit address of a variable with **la**

```
    la t1, x # loads the ADDRESS of x into t1
t1 ← will now contain 4: The address of variable x
```

- And then use that address to access memory, e.g.:

```
    la t2, 0(t1) # the contents of x into t2
```

Addr	Val
0	04
1	00
2	00
3	00
4	DE
5	C0
6	EF
7	BE
8	6C
9	34
A	00
B	01
C	02

x

Read, modify, write

- you now know enough to increment x!
- But first, lets look at some assembly

- first we **load x into a register**
- then…
- and then…

```
lw   t0, x
add  t0, t0, 1
sw   t0, x
```

- let's see what values are in **t0** and memory after this program runs

It really is that simple

- variables in asm aren't THAT scary
- please don't be afraid of them
- you just gotta remember to store if you wanna change em

Questions?

- Just in case I prepared some for you:
  - Does load word (lw) put or get data from memory?
  - I already know the word is the most “comfortable” size for the CPU, but are they the only size it can work with?

Smaller values

Smaller numeric

- MIPS also understands smaller and tiny datatypes

```
.data
    x: .word 4    =>  0x00000004
    y: .half 4    =>  0x0004
    z: .byte 4    =>  0x04
```

MIPS ISA: loading and storing 8/16-bit values

- to load/store **bytes**, we use **lb/sb**
- to load/store 16-bit (**half-word**) values, we use **lh/sh**
- these look and work just like **lw/sw,** like:

```
lb t0, tiny # loads a byte into t0
sb t0, tiny # stores a byte into tiny
```

  - ...or DO THEY?!?!?!?
- how big are registers?
  - what should go in those extra 16/24 bits then?
    - ???

can I get an extension? ...no

- sometimes you need to *widen* a number with fewer bits to more
- **zero extension** is easy: **put 0s at the beginning.**

$$1001_2 \rightarrow \textit{to 8 bits} \rightarrow 0000\ 1001_2$$

- but there are also **signed numbers** which we didn't talk about yet
  - the **top bit (MSB)** of signed numbers is **the sign (+/-)**
- **sign extension** puts ***copies of the sign bit*** at the beginning

$$1001_2 \rightarrow \textit{to 8 bits} \rightarrow 1111\ 1001_2$$

$$0010_2 \rightarrow \textit{to 8 bits} \rightarrow 0000\ 0010_2$$

  - like spreading peanut butter
    - we'll learn about *why* this is important later in the course

EXPAND VALUE

- if you load a **byte…**



10010000

31			0
00000000	00000000	00000000	00000000

If the byte is **signed…** what *should* it become?

31			0
11111111	11111111	11111111	10010000

`lb` does sign extension.

If the byte is **unsigned…** what *should* it become?

31			0
00000000	00000000	00000000	10010000

`lbu` does zero extension.

How does the CPU know whether it's signed or unsigned

- ➔ Everything's a number
- ➔ Everything's in binary (and hex is convenient shorthand)
- ➔ Numbers may not be numbers
- ➔ So, how does the computer know a number is a number?
  - How does it know that a number is signed?
  - How does it know how to add two numbers?
  - How does it know how to manipulate strings?
  - How does it know if one pattern of bits is a string or a number or a video or a program or a file or an icon or

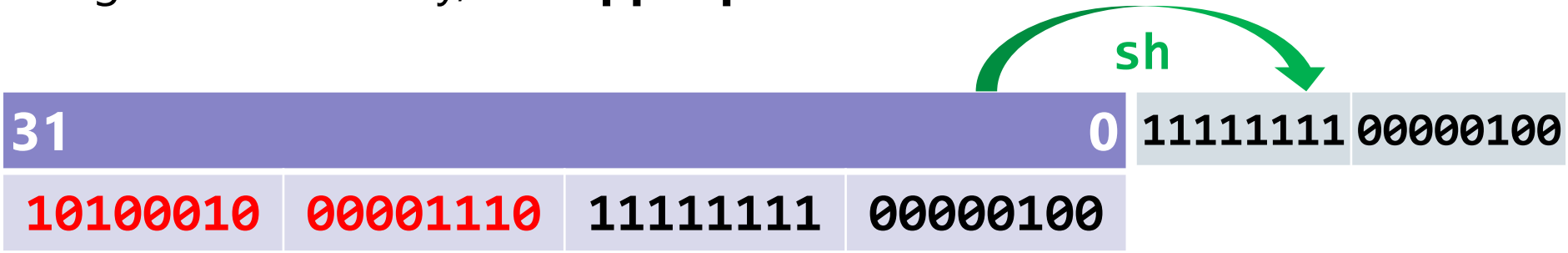
IT DOESN'T

How does the CPU know whether it's signed or unsigned

- Do YOU think the CPU knows this?
  - *no*
    - **it doesn't**
      - ***you have to use the right instruction.***
- It's particularly easy to mess this up
  - **lbu** is *usually* what you want for byte variables but **lb** is one character shorter and just looks so nice and consistent…
  - But don't!

Truncation

- If we go the other way, **the upper part of the value is cut off.**



- The sign issue doesn't exist when storing, cause we're going from a *larger* number of bits to a *smaller* number
  - therefore, **there are no sbu/shu instructions**

Memory

What is the memory?

- The **system memory** is a piece of *temporary* storage hardware
  - it's smaller and faster (more expensive!) than the *persistent* storage.
    - maybe in the future it won't be temporary
    - the line between system memory and persistent storage will fade away…
- It's where the **programs and data** that the computer is currently executing and using reside
  - all the variables, all the functions, all the open files etc.
  - the CPU **can only run programs from system memory!**

Bytes, bytes, bytes

- The memory is **a big one-dimensional array of bytes**
- What do these bytes mean?
  - ¯_(ツ)_/¯
- Every byte value has an **address**
  - This is its "array index"
  - Addresses start at 0, like arrays in C/Java
    - Gee wonder where they got the idea
    - Addresses are the offset from the beginning!
- When each byte has its own address, we call it a **byte-addressable machine**
  - not many *non*-byte-addressable machines these days

Addr	Val
0	00
1	30
2	04
3	00
4	DE
5	C0
6	EF
7	BE
8	6C
9	34
A	00
B	01
C	02

How much memory?

- Each address refers to *one* byte. if your addresses are *n* bits long… **how many bytes** can your memory have?
  - **2^n B**
- machines with 32-bit addresses can access 2^{32} B = **4GiB** of memory
  - with 64-bit addresses… **16EiB**

- Remember:
  - kibi, Mebi, Gibi, Tebi, Pebi, Exbi are powers of 2
    - **kiB** = 2^{10}, **MiB** = 2^{20}, **GiB** = 2^{30} etc.
  - kilo, mega, giga, tera, peta, exa are *ostensibly* powers of 10
    - **kB** = 10^3, **MB** = 10^6, **GB** = 10^9 etc.

Words, words, words

- For most things, we want to use **words**
  - The "comfortable" integer size for the CPU
  - On this version of MIPS, it's **32b** (**4B**)
- But our memory only holds bytes…
- **Combine multiple bytes into larger values**
  - The CPU can handle this for us
  - But importantly, *the data is still just bytes*
- When we talk about values bigger than a byte…
  - The **address** is **the address of their first byte**
    - The byte at the *smallest* address
  - So what are the addresses of the three words here?

Addr	Val
0	00
1	30
2	04
3	00
4	DE
5	C0
6	EF
7	BE
8	6C
9	34
A	00
B	01
C	02

Endianness

A matter of perspective

- let's say there's a word at address 4... made of 4 bytes
- *wh...what word do those 4 bytes represent?*

...is it
0xDEC0EFBE?

Addr	Val
...	...
7	DE
6	C0
5	EF
4	BE
...	...

...is it
0xBEEFC0DE?

Endianness

- when interpreting a *sequence of bytes* as larger values, **endianness** is the rule used to decide **what order to put the bytes in**

little-endian means the "**LITTLE** address" contains the **END-byte**

0	*1*	*2*	*3*
DE	C0	EF	BE

big-endian means the "**BIG** address" contains the **END-byte**

0xBEEFC0DE

0xDEC0EFBE

nothing to do with ***value*** of bytes, only ***order***

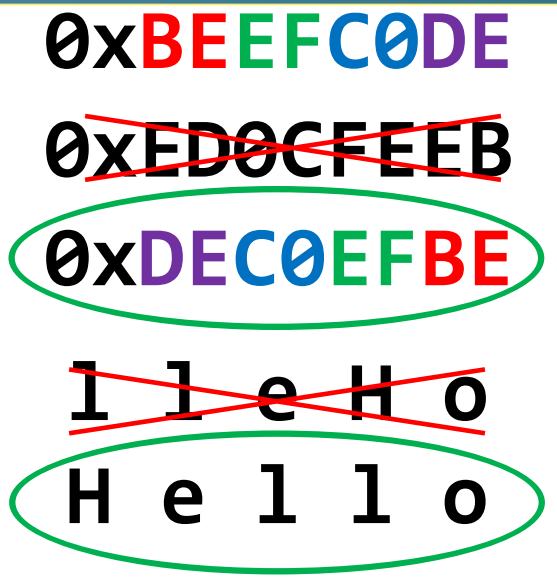
Which is better: little or big?

- it doesn't matter.* **as long as you're consistent,** it's fine
- for political (x86) reasons, most computers today are little-endian
- but endianness pops up whenever you have sequences of bytes:
  - like in files
  - or networks
  - or hardware buses
  - or… memory!
- which one is MIPS?
  - it's *bi-endian,* meaning it can be configured to work either way
  - **but** MARS uses the **endianness of the computer it's running on**
    - so **little-endian** for virtually everyone
      - cause x86
      - Apple sillycone will use a bi-endien architecture: ARM architecture

*big endian is better

What DOESN'T endianness affect?

- × the arrangement of the bits ***within a byte***
  - it just changes **meaning of order of the bytes**
    - note the bytes are still **`DE`**, **`C0`** etc.
- × **1-byte** values, arrays of bytes, **ASCII** strings…
  - *single bytes* don't care about endianness at all
- × the ordering of bytes **inside the CPU**
  - there's no need for e.g. "big-endian" arithmetic
  - the CPU works with whole *words*
- endianness only affects moving/splitting data:
  - larger than single bytes
  - between **the CPU and memory**
  - or between **multiple computers**



Summary